Learning the Cost of Rerouting: Disruption-Aware Elephant-Flow Load Balancing in SDN

EE-353 Computer Networks, Stage 2: Milestone Report (15 November 2026)

Name	CMS ID	Section
Mutahar	540395	BESE-16 A
Uswa Fatima	575207	BESE-16 A

Instructor: Ali Hassan. Source code and all raw results: https://github.com/MuhammadMuthar/sdn-reroute

1. Progress

Everything planned for this milestone is built and measured, and the first version of our improvement also runs. Done so far: (1) a frozen test environment (Mininet, Open vSwitch and os-ken 4.2.2, the maintained fork of the Ryu controller, which no longer runs on current Python); (2) a leaf-spine test network with four paths of unequal delay; (3) the ECMP baseline; (4) a Hedera-style rerouting baseline and a cooldown-only variant; (5) a dataset of 60 deliberate reroutes, each with a matched run where the flow was not moved; (6) two damage predictors (a hand-written formula and a machine-learning model) and the cost-aware controller that uses them. All five controllers run through one experiment runner on the same seeded traffic.

2. Refined problem

ECMP places each flow on a path by hashing, so two large ("elephant") flows can land on the same link while another link sits empty. A Hedera-style controller fixes this by moving an elephant to a less-loaded path. The proposal's question was what that move costs: packets arrive out of order, TCP may retransmit and slow down, and an eager controller may move a flow back and forth. We ask (a) how much damage one reroute causes and whether it can be predicted from what the controller already sees, and (b) whether a controller that moves a flow only when the expected gain is larger than the predicted damage keeps Hedera's benefit.

Our first measurements changed how we define damage. Moving a flow onto a path that already carries another elephant did not hurt the moved flow; it took share from the flow already there, and the two together lost almost nothing. So damage is now measured on the whole destination path (moved flow plus the flows already on it), as goodput lost against the matched no-move run, in the 2, 5 and 10 seconds after the move.

3. Features and initial implementation

Network. 2 leaf and 4 spine switches, 8 hosts, 10 Mbit/s links, one-way spine delays of 1, 1, 5 and 10 ms so that a reroute really reorders packets. Only the ingress leaf chooses the path, so a reroute is a single rule change and no packet is dropped by the change itself. **Controllers** (Python, OpenFlow 1.3, one shared base): ECMP with select groups; Hedera-style (polls flow statistics every second, treats a flow above 10 % of link capacity as an elephant, moves it to the least-loaded path); cooldown-only (the same plus a 5 s per-flow cooldown and a fixed margin); cost-aware with the formula; cost-aware with the ML model. **Traffic.** Seeded and repeatable: 4 iperf3 elephants plus about 1050 short flows with heavy-tailed sizes per 30 s run. **Measurement.** Per-flow goodput, short-flow completion time, link utilisation, kernel TCP counters (retransmissions, reordering, timeouts), per-socket statistics every 50 ms, and packet captures that show which path each flow really took. A replay tool reruns any reroute with and without the move. 31 unit tests cover the analysis and the decision rule.

4. Baseline implemented and tested: ECMP

ECMP was run 10 times with seeds 1 to 10. Group statistics and the captures confirm that flows spread over all four paths and that the same seed gives the same traffic. The weakness shows clearly: with only four elephants the hash often puts two or three on one uplink. In an earlier set of 10 runs an elephant alone on its uplink got 8.70 Mbit/s, one sharing with another got 4.42 Mbit/s and one sharing with two others got 3.09 Mbit/s. In the comparison runs below, the average elephant under ECMP got 5.94 ± 1.04 Mbit/s and the worst one 4.60 ± 1.09 Mbit/s, on links that could give each of them close to 10.

5. Initial results and preliminary comparison

Controller (10 runs each, mean ± 95 % CI)	Elephant goodput, mean (Mbit/s)	Worst elephant (Mbit/s)	Short-flow time, median (ms)	Short-flow time, 99th pct (ms)	Reroutes per minute
ECMP (baseline)	5.94 ± 1.04	4.60 ± 1.09	50.5 ± 6.7	1157 ± 63	0
Hedera-style (baseline)	8.54 ± 0.11	8.33 ± 0.16	63.3 ± 1.8	1131 ± 32	3.8 ± 1.6
Cooldown-only	8.49 ± 0.11	8.31 ± 0.16	63.4 ± 2.4	1113 ± 16	4.8 ± 2.4
Cost-aware, formula (ours)	8.48 ± 0.19	8.30 ± 0.22	63.3 ± 1.7	1119 ± 27	5.2 ± 2.5
Cost-aware, ML (ours)	8.35 ± 0.22	8.16 ± 0.27	63.0 ± 2.2	1130 ± 17	7.6 ± 3.3

Table 1. All controllers on identical traffic (same seeds, same machine boot). Short-flow time is flow completion time.

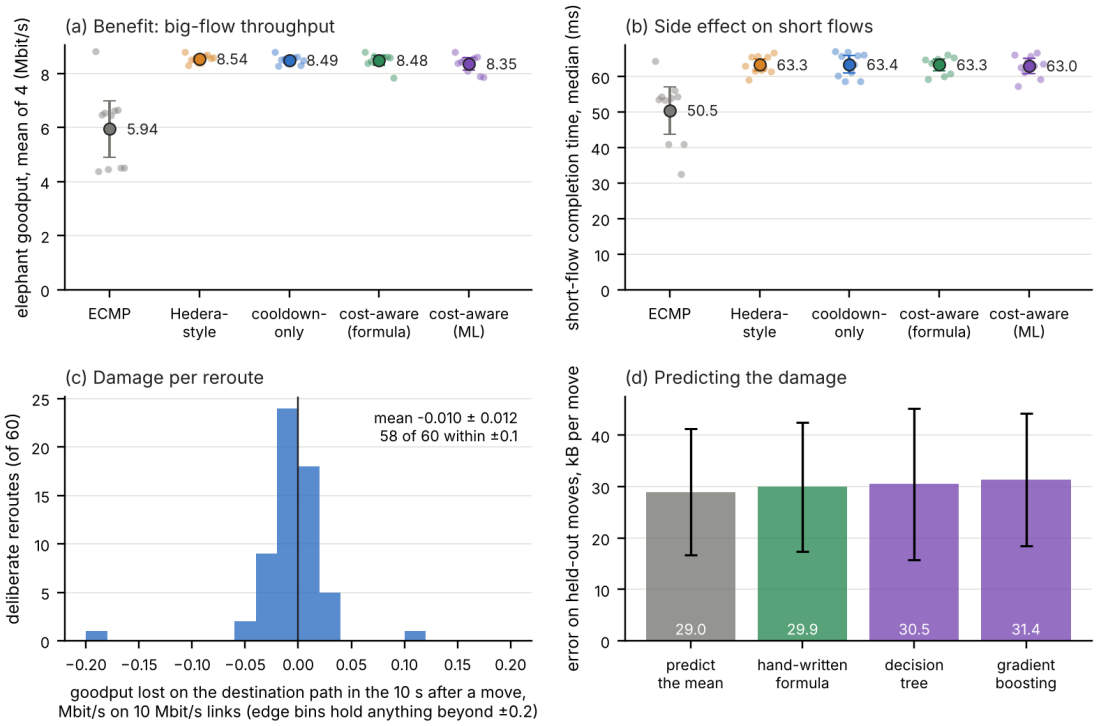


Figure 1. (a), (b): one small dot per run, large dot and bar are the mean and 95 % confidence interval. (c): the 60 deliberate reroutes; positive means the move cost goodput. (d): prediction error on moves the model was not trained on (10 held-out folds).

Rerouting beats ECMP. Paired by seed, Hedera-style rerouting gives each elephant 2.59 Mbit/s more than ECMP (95 % CI 1.62 to 3.56), and the worst elephant goes from 4.60 to 8.33 Mbit/s. **It has a price for short flows.** Their median completion time rises by 12.8 ms (CI 5.6 to 20.0); the 99th percentile shows no clear change. Our reading is that once the elephants are spread out, every path carries one, so no path is left empty for short flows. We also found a side effect of monitoring itself: the per-flow counting rules send the first packets of a short flow through the switch's slow path, which raises reordering events from about 3 to about 1000 per run (confirmed with Open vSwitch's own counters). It did not slow the short flows measurably, but we report it.

The damage of one reroute is small here. Moves do disturb TCP: the moved flow's socket saw reordering in 24 of 60 moves and in none of the 60 no-move runs. But the goodput lost on the destination path over 10 s is -0.010 ± 0.012 Mbit/s, which is no measurable loss, and 58 of 60 moves are within ±0.1 Mbit/s. Modern Linux TCP detects reordering and undoes the unnecessary slow-down quickly. For comparison, rerouting gains each elephant 2.59 Mbit/s.

6. The improvement, and where it stands

The rule. Move flow *f* from path *p* to path *q* only if the bytes it is expected to gain over the next 10 s are more than the predicted damage *C(f, p, q)* in bytes, and *f* has not been moved in the last 5 s. *C* is predicted from what the controller already logs: flow rate, load and number of elephants on both paths, delay difference, flow age, bytes sent and earlier moves. We built two predictors, a hand-written formula with fitted coefficients and a gradient-boosted tree (scikit-learn), trained on the 60 deliberate reroutes plus 30 moves the controllers made themselves. A decision takes 0.19 ms with the ML model and 0.002 ms with the formula.

Honest first result. Both cost-aware controllers keep the gain over ECMP (+2.54 and +2.41 Mbit/s per elephant) but are not better than plain Hedera-style rerouting: the formula version differs from it by -0.05 Mbit/s (CI -0.23 to 0.13), and the ML version makes more reroutes, not fewer (+3.8 per minute, CI 1.2 to 6.4). The reason is in Fig. 1(c) and (d). On this network there is almost no damage to predict, so neither predictor beats simply guessing the average (error 29.0 kB per move for the average, 29.9 for the formula, 31.4 for the ML model). We state this plainly: at 10 Mbit/s with 1 to 10 ms paths, our improvement does not yet improve on Hedera.

Next, for Stage 3. A 16-trial pilot at 100 Mbit/s shows 10 to 50 times more reordering per move, and moves from a fast path onto an empty slow path cost 6 to 12 % of the link for 2 s (only 2 such trials so far). We will therefore (1) test the conditions where damage should matter (higher link rates, larger delay gaps, frequent moves) and retrain there, (2) add the fat-tree network, a link that degrades mid-run and the synchronized AI-training traffic from the proposal, and (3) report whichever way it comes out. If damage stays small, the finding is that reroute damage is not what limits Hedera-style controllers on modern Linux TCP.

7. Source code

https://github.com/MuhammadMuthar/sdn-reroute (public). README.md gives install and run commands; every number and figure in this report is produced by a script in the repository from the raw results stored there (*analysis/stage2_report.py* for Table 1 and Figure 1). Experiments were run on a native Ubuntu laptop; nothing here is simulated or estimated.